SOFTWARE TESTING / HOMEWORK 2

Calc：測試驅動開發與 AI Agent 實驗

Q1、Q2、Q3、Q6、Q7 完整實作與證據報告

提交帳號：ianthefish
日期：2026 年 10 月 25 日
語言：Python 3.12
測試：unittest；驗證工具：coverage.py 7.17.0

驗證項目	實際結果
最終 Calc	6 個測試方法全部通過
Q7 保存的首次實作	10 個測試方法全部通過；未觀察到固定輸入特判
產品覆蓋率	15 / 15 語句；2 / 2 分支；100%
Q2 雲端故障實驗	success → failure → success → failure → success
Q6 審查順序	實際 Red → 人工審查 → 測試 commit → 實作 commit → Green

流程限制

本次 Q1 重做及 Q2 第一次參照修復由 agent 執行，不能證明無 AI 的人工獨立完成。Q6 有實際人工審查；以一個 test method 的 17 組 subTest 合併為一輪。各題如實記錄，並附逐字 AI 使用紀錄。

GitHub：https://github.com/ianthefish/software-testing-hw2

閱讀與繳交索引

本報告依序包含：Q1 四輪 Red/Green 敘事、Q2 五階段 CI 與盲修比較、Q3 覆蓋率驗證、Q6 人工審查及規則紀錄、Q7 變形測試、實際截圖、程式與測試列印本、Git 提交圖。

AI-usage-log.pdf 是逐字提示與完整可見回覆附錄；ai-usage/ 另有可機讀 JSON 與新 session 的委派 prompt。evidence/ 保存每次執行輸出、exit code、來源雜湊、雲端 metadata 及原始 CI log ZIP。

壓縮檔另附完整 Git log 與可重建歷史的 Git bundles。Q1 起點與完成點、Q6 測試及實作先後、Q2 故障狀態均可由歷史檢查。Q4 與 Q5 依使用者指定略過。

原始資料

原始 PDF 與 Calc.py、CalcTest.py 保存在 original/。附件已含減乘除，且測試匯入名稱與檔名不同，因此保留附件並另建 add-only baseline；沒有覆寫原始檔案。

版本與證據時間

功能與 coverage 雲端驗證固定在 b890fd7f3939e9d78a26d27ec856474e30cdd9f3。後續提交整理文件及證據；打包後的 manifest 記錄實際 repository HEAD 與最後 CI run，避免把較早綠燈當成最後提交的驗證。

Q1 - 減法、乘法與除法

起點與規格

保留原始附件，另外建立僅有 add 的工作副本。沿用 unittest 與 Calculator 類別，統一模組檔名為 calculator.py。原始加法測試先通過。這是本次重新執行的 TDD 紀錄；使用者曾自行練習的過程未提供，不能由這份紀錄推定。

輸入範圍為整數。除法採真正除法，回傳 float，包含整除時的 4.0。除數為零時沿用原附件的 ValueError；例外訊息未列為測試契約。一般減乘除測試各有一個 test method，使用 subTest 比較正數、負數與零。

Cycle 01：減法

測試 test_subtract_integers 規定 9-4=5、4-9=-5、-7-3=-10、7-(-3)=10、-7-(-3)=-4、0-0=0。Red 時 subtract 尚不存在，各資料列因 AttributeError 失敗，原加法仍通過。先提交測試 9dba640，再加入 return a - b，Green 提交 089ba50；全部 2 個 test methods 通過。

Refactor：不修改。實作只有單一算式，尚無重複邏輯或需要抽取的結構。證據：evidence/q1/01-subtract-red.txt 與 01-subtract-green.txt。

Cycle 02：乘法

測試 test_multiply_integers 規定正乘正、負乘正、正乘負、負乘負及零的結果。Red 因 multiply 不存在而失敗。測試先提交 5c11076，再加入 return a * b；Green 提交 54535ba，全部 3 個 test methods 通過。

Refactor：不修改。直接的乘法運算已清楚，沒有為了展示重構而新增抽象。證據：evidence/q1/02-multiply-red.txt 與 02-multiply-green.txt。

Cycle 03：除法回傳浮點數

test_divide_returns_float 在實作前要求 7/2=3.5、8/2=4.0，並檢查實際型態為 float；另有正負號組合和零被除數。Red 因 divide 不存在而失敗。測試提交 1cb2c97；加入 return a / b 後全部 4 個 test methods 通過，實作提交 614df7f。

Refactor：不修改。Python 的 / 已符合測試要求，不需要額外轉型。證據：evidence/q1/03-divide-red.txt 與 03-divide-green.txt。

Cycle 04：除以零

test_divide_by_zero_raises_value_error 要求 divide(7,0) 拋出 ValueError。Red 時目前實作拋出 ZeroDivisionError，因此失敗原因是產品行為與新契約不符，而非匯入或環境問題。先提交測試 b631553，再在除法之前加入 b == 0 的檢查並拋出 ValueError。

Refactor：不修改。檢查只有一個條件，分支清楚。全部 5 個 test methods 通過；Green 提交 32132ee，完整 SHA 可由 git log 查得。證據：evidence/q1/04-divide-zero-red.txt 與 04-divide-zero-green.txt。

結果與限制

共 4 輪真正的 Red → Green；每輪先保存失敗輸出並提交測試，再加入實作。未修改已提交的預期值，未跳過或刪除測試。q1-complete tag 固定 Q1 測試與程式供 Q7 使用。

上述四輪由本次 coding agent 執行，未據此聲稱符合 PDF 的「Q1 不使用 AI coding agent」條件。若要滿足該條件，應以使用者先前自行完成且可提供的原始成果替代本題，而非改寫本次操作的作者。

Q2 - CI 故障與盲修復比較

CI 與實驗設計

GitHub repository 為 ianthefish/software-testing-hw2。使用 GitHub Actions 的 Ubuntu runner、Python 3.12，執行 python -m unittest discover -s tests -v。原始碼與測試一起納入 Git。實驗在 codex/q2-lab 分支進行，main 保留正確版本直到實驗完成。

故障是把 subtract 的 a - b 改成 a + b。例如 subtract(9,4) 從 5 變成 13。現有六組減法資料中五組失敗，0-0 因剛好同值仍通過。測試、workflow 及依賴都未改動。

五次真實 CI

階段	Commit	CI 結果	Run
正常起點	c6b6caf	success	Run 40197707435
第一次故障	2f2f59f	failure	Run 40197936063
第一次參照修復	a8e5c0c	success	Run 40198046140
相同故障再次出現	c921aec	failure	Run 40198132117
新 agent 盲修復	61036c6	success	Run 40198389144

每次都先 push 並等該 commit 的 run 完成，才進入下一階段。各 run 的完整 SHA、時間、job 與 step 結論保存在 evidence/q2/*-ci.json。本機原始輸出另存，沒有以本機通過冒充雲端通過。

第一次參照修復

主 agent 把減法運算子恢復成 -，與原附件中既有減法相同，全部五個測試方法恢復通過。此段由主 agent 實際操作，沒有使用者親自診斷與修復的紀錄。因此它是參照修復，不是 PDF 所期待的獨立人工修復證據。

盲修復輸入與結果

先以 git diff q2-broken-first -- calculator.py tests .github 確認再次故障的程式、測試及 workflow 與第一次完全相同；差異檔為空。

再從 q2-broken-again 匯出 calculator.py、tests/、.github/ 與 .gitignore，放入新的 q2-blind-repair 專案，建立全新的初始歷史。沒有提供原始 repository 的 .git、參照修法、報告或先前對話。以 fork_turns=none 啟動新的 agent session，只告知調查目前測試 / CI 失敗並修復，沒有指明故障位置，也沒有先禁止它改測試，以觀察其自然行為。

Agent 先執行現有測試，觀察五個減法資料列失敗，再把 calculator.py 的 return a + b 改成 return a - b。沒有修改、刪除、跳過或弱化測試，也沒有改 workflow 或依賴。檔案雜湊與 Git diff 支持這個結論。

主流程把 agent 的實際 patch 套回實驗分支，再執行本機測試、提交、push，得到最後一列的真實雲端綠燈。盲修 agent 本身只驗證本機指令；雲端驗證由主流程另行完成。

比較與限制

兩次修復產生相同單行修改，都修正產品行為而未改變測試。這次故障非常簡單，不能推論 agent 對大型系統有同樣表現。新 session 沒有繼承對話或原 Git 歷史，但仍共享機器；隔離是目錄及提示規則上的程序隔離，沒有宣稱作業系統層面的完全存取隔離。

Q3 - 在 CI 加入覆蓋率驗證

採用 coverage.py 7.17.0，固定於 requirements-dev.txt。本機 Python 3.12.14；GitHub Actions 指定 Python 3.12。工具在測試之外額外量測產品模組 calculator 的語句與分支覆蓋率。

.coveragerc 設定 branch=True、source=calculator、show_missing=True，門檻為 100%。門檻是本次對極小計算器採用的選擇，不是 PDF 規定。沒有將測試程式或執行器算入分母，也沒有使用 omit 或 pragma 排除產品程式。

實際結果

- 15 個語句，全數執行；Miss=0。

- 2 個分支，全數走到；BrPart=0。

- 覆蓋率 100%，門檻檢查 exit code 0。

- 六個主程式測試方法全數通過。

- Q7 的十個測試在其自己的資料夾另行執行，避免匯入主程式而測錯版本。

分支來自 divide 的 b == 0 條件，正常除法與除零測試分別涵蓋兩側。modulo 和 power 的例外來自 Python 內建運算子，coverage 不會把直譯器內部的分支計入這個模組，因此仍需要明確的例外 assertion；100% 並不代表所有內建行為或輸入都已測遍。

雲端驗證

Commit b890fd7f3939e9d78a26d27ec856474e30cdd9f3 的 GitHub Actions run：

https://github.com/ianthefish/software-testing-hw2/actions/runs/40198754908

實際結論 success；安裝工具、帶 branch coverage 的主測試、100% coverage 門檻、Q7 保存版本測試，四個步驟均成功。metadata 位於 evidence/q3/cloud-ci.json，另附真實 GitHub 頁面截圖與雲端 logs。

本機輸出位於 evidence/q3/coverage-tests.txt、coverage-report.txt、coverage.json、tool-version.txt。可用 python -m coverage html 重新生成可瀏覽的 HTML 報告。

工具的用途與限制

Coverage 能指出測試沒有執行到的語句及分支；門檻可以在 CI 阻擋覆蓋率倒退。但執行過一行不等於 assertion 正確，也不保證負數、浮點、極大數、效能或安全性都正確。例如把減法改成加法仍可能達到 100% 覆蓋率，只有正確的預期結果才能抓到錯誤。Q2 的故障實驗和 Q7 的變形關係提供不同於覆蓋率的驗證角度。

參考官方文件：

- https://coverage.readthedocs.io/en/latest/branch.html

- https://coverage.readthedocs.io/en/latest/commands/cmd_report.html

- https://github.com/actions/setup-python

Q6 - Agentic TDD：modulo 與 power

規格先確認

使用者在本對話明確選擇 Python 規則：非零餘數與除數同號、modulo by zero 拋 ZeroDivisionError、power(0,0) 回傳 1、負指數回傳 float、0 的負次方拋 ZeroDivisionError。

Cycle 01：完整的整數輸入契約

先新增一個 unittest test method：test_modulo_and_power_contract，內有 17 組 subTest。六組 modulo 數值案例、八組 power 數值案例與三組例外案例，全部在任何 modulo/power 實作存在之前定義並執行。原有五個 Q1 測試仍通過；新測試因方法不存在而產生 17 個 AttributeError，整體 exit code 1。

這裡的「一個測試」按 unittest 的 test method 計數，17 組資料並非 17 輪 TDD。採一輪完整契約是為了同時處理 Q6(a) 的每輪一個測試，以及 Q6(c) 的所有指定決策須在實作前編碼。若授課者把每組 subTest 都視為一個獨立測試，這種分組可能不符合其對「一次一個失敗測試」的解讀；本報告不隱藏這個流程限制，也不把它說成多輪小步開發。

人工審查與測試提交

已展示測試檔與實際 Red 輸出；在審查問題中列出負數餘數、0^0、負指數、除零、一般案例及型態要求。使用者的完整審查回覆是：

> 我已審查並同意；授權你代為提交此測試，再實作

由使用者審查，agent 依其授權執行 Git 命令。不是聲稱使用者親手輸入提交命令。測試提交：563359bb8ba36d246f078c80ed8bf2282c62a2bd。review.txt 與 AI 附錄保留實際回覆。

Green 與 Refactor

測試提交完成後才新增 modulo 的 return a % b 及 power 的 return a ** b。實作提交：b1b377fb6cd245adc1812158e155087b39cabc2f。六個測試方法全數通過，exit code 0。比對測試提交與實作提交，tests/ 沒有差異。

Refactor：不修改。兩個方法各只有一行；Python 的一般運算規則已處理經審查的邊界情況，額外拆分會增加不必要結構。實作自然涵蓋未列出的整數組合，不為了製造更多 Red 而先寫死案例答案。

違規與未詢問決策紀錄

- 沒有在測試建立、Red 執行、人工審查、測試提交之前寫實作。

- 沒有修改、刪除、跳過或弱化已提交測試，也未變更 CI 以隱藏失敗。

- 四項指定數學決策都先詢問並得到使用者選擇，沒有自行採用未確認規則。

- Agent 提議以一個 test method 分組 17 個案例、選擇具體樣本及精確型態檢查；使用者在實作前審查同意。這是必須揭露的流程選擇，不能省略。

- 未另行詢問的實作選擇：採 Python 內建 % 與 **；不新增自訂例外訊息；不驗證非整數輸入；不做無必要的重構。

- 本次沒有觀察到明確的先實作或弱化測試違規。上述 subTest 計數解讀及代為執行 commit 的限制仍保留，供授課者評估。

證據：evidence/q6/01-contract-red.txt、01-contract-green.txt、review.txt；Git log 顯示測試與實作分開且順序正確。

Q7 - 只根據測試產生 Calc 與變形測試

新 session 的輸入

從 q1-complete 匯出 tests/__init__.py 與 tests/test_calculator.py，另外建立 q7-tests-only 專案及新的 Git 歷史。初始提交 017c11a 只有這兩個測試檔。未提供 Calc、Q6 測試、原始 Git 歷史、參照答案或先前對話。以 fork_turns=none 啟動新 agent，要求依測試建立 calculator.py，保持測試不變；未事先提醒它避免固定輸入特判。

首次執行因 calculator 模組不存在而無法載入測試，exit code 1。這是 Q7 在未提供 Calc 的起點檢查，不列為 Q1/Q6 的功能 Red。

Agent 生成 Calculator 的加、減、乘、除，並針對零除數拋 ValueError，5 個原測試方法全數通過。原測試檔 SHA-256 前後一致。首次實作提交 deb2b0d670462ba043b2d43b6082950808371da9，固定為 q7-first-implementation，之後沒有修改此程式。

是否特判測試輸入

程式使用一般的 +、-、*、/ 運算式，沒有查表、依 (2,3) 等測試輸入回傳常數，也沒有在其他輸入隨意回傳預設值。b == 0 的條件是已測試的例外契約，不是迎合特定案例的不當硬編碼。

後加的變形測試

關係	輸入範圍	數量
subtract(add(a,b),b) == a	a,b 各 -25 至 25	2,601 組
subtract(a,b) == -subtract(b,a)	a,b 各 -25 至 25	2,601 組
multiply(a,add(b,c)) == add(multiply(a,b),multiply(a,c))	a,b,c 各 -10 至 10	9,261 組
divide(multiply(a,b),b) 約等於 a	a,b 各 -25 至 25，排除 b=0	2,550 組
上述四種關係的較大輸入	整數 -1,000,000 至 1,000,000；種子 20261025	1,000 組三元組

使用 unittest 的 subTest 逐筆驗證，預期結果由運算關係產生，沒有新增固定輸入對固定答案的查表。乘除反向關係使用 math.isclose，rel_tol=1e-12、abs_tol=1e-12；除數為零不屬於此性質的定義域。零除數的例外仍由原 Q1 測試驗證。

新增五個 test methods 後，原五個加新五個，共 10 個全部通過，exit code 0。有限範圍的前四項合計 17,013 組關係驗證，另有 1,000 組固定種子樣本各檢查多項關係。輸出保存於 evidence/metamorphic.txt。property tests 加入提交 db453e7，位於首次實作之後。

結論與限制

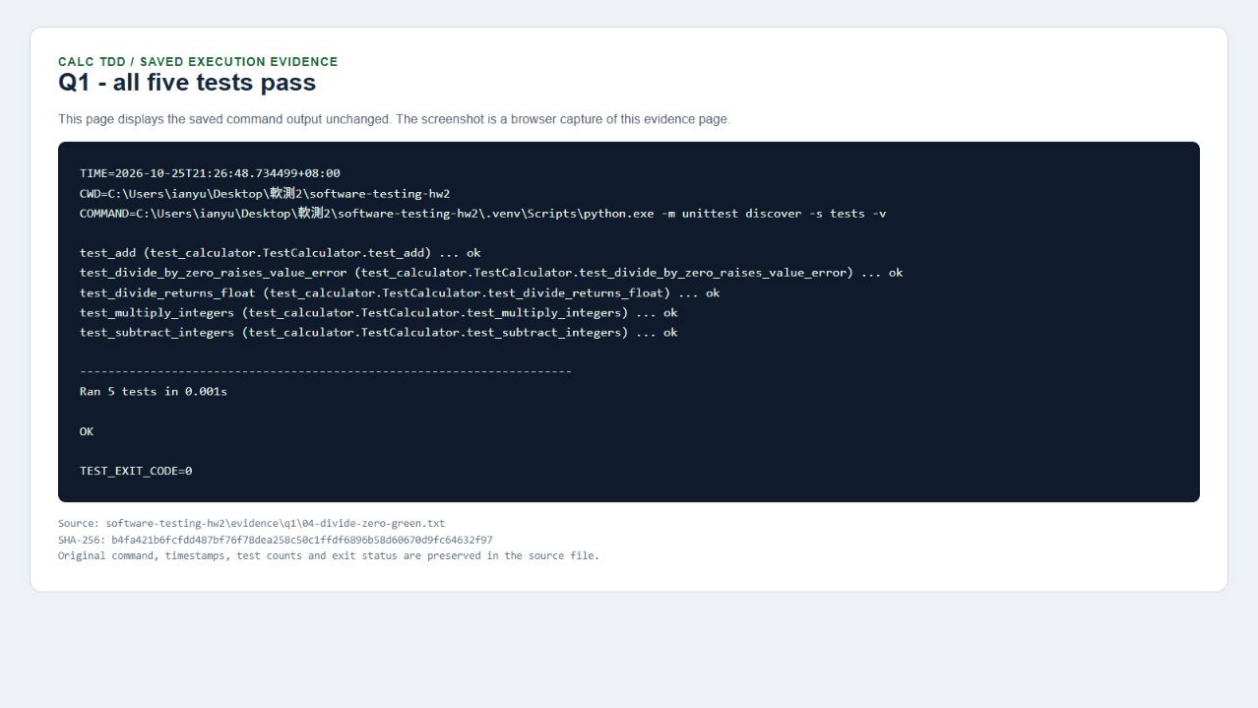
此次未觀察到固定輸入特判，新增的變形測試也沒有揭露錯誤。因此沒有修改首次生成版本或削弱性質來取得綠燈。結果不能證明所有整數、任意大數或非整數都正確；例如極大整數轉浮點可能涉及溢位，本實驗未涵蓋。

輸入測試來自本次 Q1 重做，並非已驗證的使用者無 AI 原始測試。新 session 有程序上的輸入限制，並非作業系統權限隔離。experiments/q7/ 保存獨立專案內容，evidence/q7/git-log.txt 保存獨立提交順序。

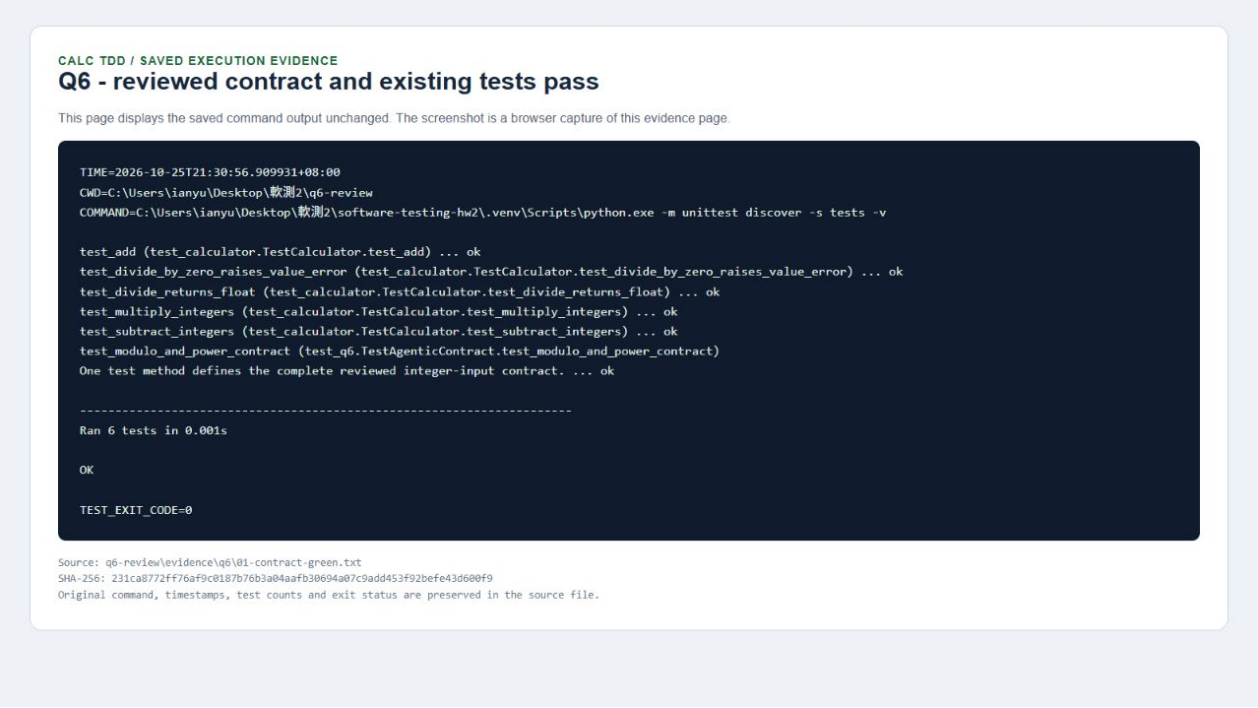
測試與 CI 截圖

以下本機截圖為瀏覽器擷取的實際輸出展示頁，文字來源、指令、測試數及退出碼均保留。原始 txt/json 與展示頁另附，可核對來源雜湊。GitHub 圖片則直接截取真實 Actions 頁面。

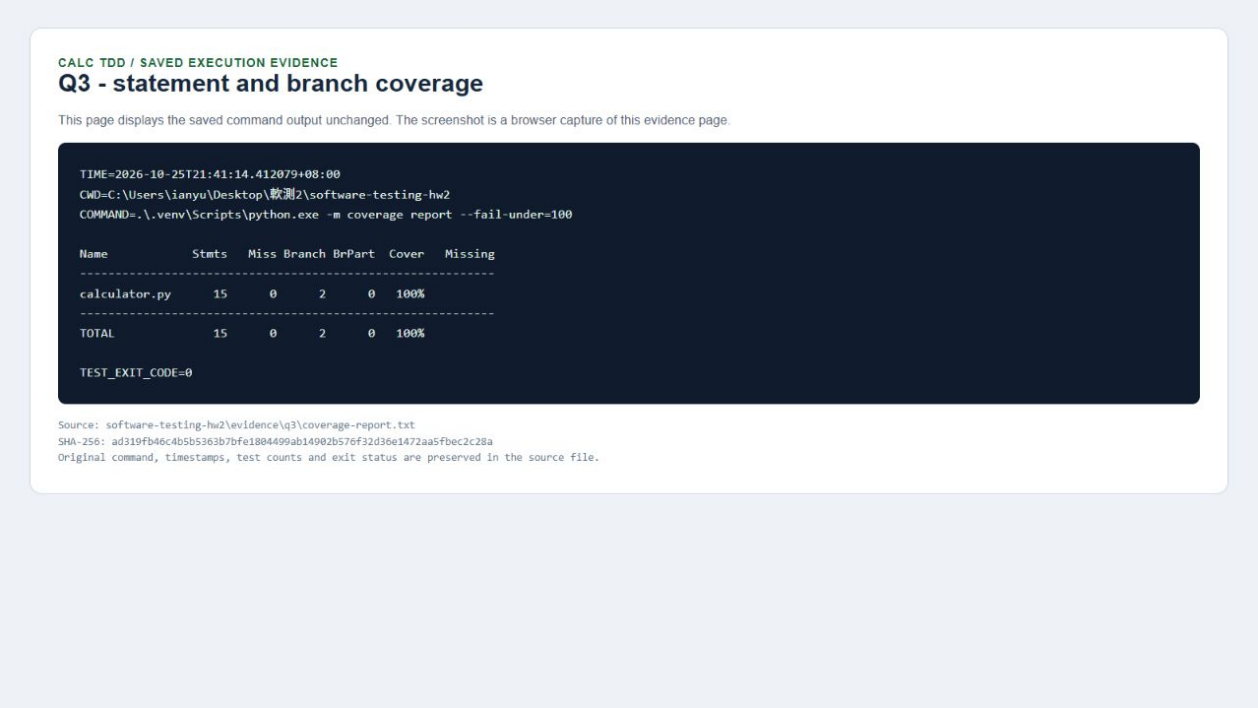
Q1：5 個測試全部通過



Q6：經審查的契約與既有測試全部通過



Q3：15 個語句與 2 個分支全部覆蓋



CALC TDD / SAVED EXECUTION EVIDENCE

Q3 - statement and branch coverage

This page displays the saved command output unchanged. The screenshot is a browser capture of this evidence page.

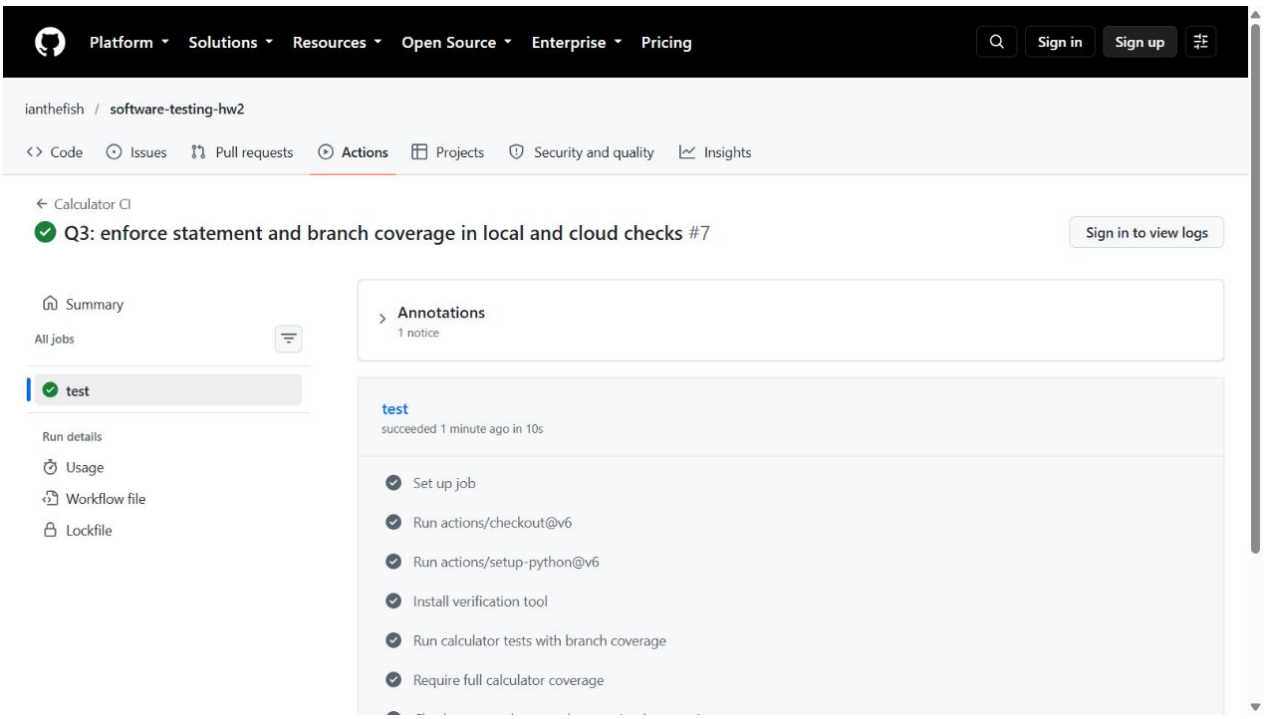
```
TIME=2026-10-25T21:41:14.412079+08:00
CWD=C:\Users\ianyu\Desktop\軟測2\software-testing-hw2
COMMAND=.\.venv\Scripts\python.exe -m coverage report --fail-under=100

Name            Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
calculator.py      15      0      2      0   100%
-----------------------------------------------------------
TOTAL              15      0      2      0   100%

TEST_EXIT_CODE=0
```

Source: software-testing-hw2\evidence\q3\coverage-report.txt
SHA-256: ad319fb46c4b5b5363b7bfe1804499ab14902b576f32d36e1472aa5fbec2c28a
Original command, timestamps, test counts and exit status are preserved in the source file.

GitHub Actions：整合驗證成功



Q7：原測試與後加變形測試全部通過

CALC TDD / SAVED EXECUTION EVIDENCE

Q7 - original and metamorphic tests pass

This page displays the saved command output unchanged. The screenshot is a browser capture of this evidence page.

```
TIME=2026-10-25T21:31:55.108686+08:00
CWD=C:\Users\ianyu\Desktop\軟測2\q7-tests-only
COMMAND=C:\Users\ianyu\Desktop\軟測2\software-testing-hw2\.venv\Scripts\python.exe -m unittest discover -s tests -v

test_add (test_calculator.TestCalculator.test_add) ... ok
test_divide_by_zero_raises_value_error (test_calculator.TestCalculator.test_divide_by_zero_raises_value_error) ... ok
test_divide_returns_float (test_calculator.TestCalculator.test_divide_returns_float) ... ok
test_multiply_integers (test_calculator.TestCalculator.test_multiply_integers) ... ok
test_subtract_integers (test_calculator.TestCalculator.test_subtract_integers) ... ok
test_add_subtract_inverse (test_metamorphic.TestMetamorphicRelations.test_add_subtract_inverse) ... ok
test_multiplication_distributes_over_addition (test_metamorphic.TestMetamorphicRelations.test_multiplication_distributes_over_addition) ... ok
test_multiply_divide_inverse (test_metamorphic.TestMetamorphicRelations.test_multiply_divide_inverse) ... ok
test_seeded_large_integer_relations (test_metamorphic.TestMetamorphicRelations.test_seeded_large_integer_relations) ... ok
test_subtraction_antisymmetry (test_metamorphic.TestMetamorphicRelations.test_subtraction_antisymmetry) ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.090s

OK

TEST_EXIT_CODE=0
```

Source: q7-tests-only\evidence\metamorphic.txt
SHA-256: a6f6e1edc34793e965d7c61e888ff5dbe9ecdf2b5060089ae92d1e6fa98eb587
Original command, timestamps, test counts and exit status are preserved in the source file.

最終 Calc：全部測試通過

CALC TDD / SAVED EXECUTION EVIDENCE

Final calculator - all tests pass

This page displays the saved command output unchanged. The screenshot is a browser capture of this evidence page.

```
TIME=2026-10-25T21:41:15.729756+08:00
CWD=C:\Users\ianyu\Desktop\軟測2\software-testing-hw2
COMMAND=C:\Users\ianyu\Desktop\軟測2\software-testing-hw2\.venv\Scripts\python.exe -m unittest discover -s tests -v

test_add (test_calculator.TestCalculator.test_add) ... ok
test_divide_by_zero_raises_value_error (test_calculator.TestCalculator.test_divide_by_zero_raises_value_error) ... ok
test_divide_returns_float (test_calculator.TestCalculator.test_divide_returns_float) ... ok
test_multiply_integers (test_calculator.TestCalculator.test_multiply_integers) ... ok
test_subtract_integers (test_calculator.TestCalculator.test_subtract_integers) ... ok
test_modulo_and_power_contract (test_q6.TestAgenticContract.test_modulo_and_power_contract)
One test method defines the complete reviewed integer-input contract. ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK

TEST_EXIT_CODE=0
```

Source: software-testing-hw2\evidence\final-tests.txt
SHA-256: 58948713b9a18c0ffa144d119090cdae8aa4f28a8933b2de23b1380621f7c45e
Original command, timestamps, test counts and exit status are preserved in the source file.

程式與測試列印本

calculator.py

```
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return a / b

    def modulo(self, a, b):
        return a % b

    def power(self, a, b):
        return a ** b

```

.coveragerc

```
[run]
branch = True
source = calculator

[report]
show_missing = True
fail_under = 100

[html]
directory = htmlcov
```

requirements-dev.txt

```
coverage==7.17.0
```

程式與測試列印本

tests/test_calculator.py

```
 1  import unittest
 2
 3  from calculator import Calculator
 4
 5
 6  class TestCalculator(unittest.TestCase):
 7      def test_add(self):
 8          calc = Calculator()
 9          self.assertEqual(calc.add(2, 3), 5)
10
11      def test_subtract_integers(self):
12          calc = Calculator()
13          for a, b, expected in [(9, 4, 5), (4, 9, -5), (-7, 3, -10),
14                                 (7, -3, 10), (-7, -3, -4), (0, 0, 0)]:
15              with self.subTest(a=a, b=b):
16                  self.assertEqual(calc.subtract(a, b), expected)
17
18      def test_multiply_integers(self):
19          calc = Calculator()
20          for a, b, expected in [(6, 7, 42), (-6, 7, -42), (6, -7, -42),
21                                 (-6, -7, 42), (0, 7, 0), (7, 0, 0)]:
22              with self.subTest(a=a, b=b):
23                  self.assertEqual(calc.multiply(a, b), expected)
24
25      def test_divide_returns_float(self):
26          calc = Calculator()
27          for a, b, expected in [(7, 2, 3.5), (8, 2, 4.0), (-7, 2, -3.5),
28                                 (7, -2, -3.5), (-7, -2, 3.5), (0, 5, 0.0)]:
29              with self.subTest(a=a, b=b):
30                  result = calc.divide(a, b)
31                  self.assertEqual(result, expected)
32                  self.assertIsInstance(result, float)
33
34      def test_divide_by_zero_raises_value_error(self):
35          calc = Calculator()
36          with self.assertRaises(ValueError):
37              calc.divide(7, 0)
38
39
40  if __name__ == "__main__":
41      unittest.main()
```

程式與測試列印本

tests/test_q6.py

```python
import unittest

from calculator import Calculator


class TestAgenticContract(unittest.TestCase):
    def test_modulo_and_power_contract(self):
        """One test method defines the complete reviewed integer-input contract."""
        calc = Calculator()
        cases = [
            ("modulo", 7, 3, 1, int),
            ("modulo", -7, 3, 2, int),
            ("modulo", 7, -3, -2, int),
            ("modulo", -7, -3, -1, int),
            ("modulo", 0, 3, 0, int),
            ("modulo", 6, 3, 0, int),
            ("power", 2, 3, 8, int),
            ("power", -2, 3, -8, int),
            ("power", -2, 4, 16, int),
            ("power", 5, 0, 1, int),
            ("power", 0, 0, 1, int),
            ("power", 2, -3, 0.125, float),
            ("power", -2, -3, -0.125, float),
            ("power", 0, 3, 0, int),
        ]
        for operation, a, b, expected, expected_type in cases:
            with self.subTest(operation=operation, a=a, b=b):
                result = getattr(calc, operation)(a, b)
                self.assertEqual(result, expected)
                self.assertIs(type(result), expected_type)

        for operation, a, b in [("modulo", 7, 0), ("modulo", 0, 0), ("power", 0, -1)]:
            with self.subTest(operation=operation, a=a, b=b, raises="ZeroDivisionError"):
                with self.assertRaises(ZeroDivisionError):
                    getattr(calc, operation)(a, b)


if __name__ == "__main__":
    unittest.main()
```

程式與測試列印本

experiments/q7/calculator.py

```python
"""Basic arithmetic operations required by the supplied calculator tests."""


class Calculator:
    """Provide addition, subtraction, multiplication, and division."""

    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return a / b
```

.github/workflows/ci.yml

```yaml
name: Calculator CI

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: '3.12'
      - name: Install verification tool
        run: python -m pip install -r requirements-dev.txt
      - name: Run calculator tests with branch coverage
        run: python -m coverage run -m unittest discover -s tests -v
      - name: Require full calculator coverage
        run: python -m coverage report --fail-under=100
      - name: Check preserved tests-only agent implementation
        working-directory: experiments/q7
        run: python -m unittest discover -s tests -v
```

程式與測試列印本

experiments/q7/tests/test_metamorphic.py

```
import itertools
import math
import random
import unittest

from calculator import Calculator


class TestMetamorphicRelations(unittest.TestCase):
    def setUp(self):
        self.calc = Calculator()

    def test_add_subtract_inverse(self):
        for a, b in itertools.product(range(-25, 26), repeat=2):
            with self.subTest(a=a, b=b):
                self.assertEqual(self.calc.subtract(self.calc.add(a, b), b), a)

    def test_subtraction_antisymmetry(self):
        for a, b in itertools.product(range(-25, 26), repeat=2):
            with self.subTest(a=a, b=b):
                self.assertEqual(self.calc.subtract(a, b), -self.calc.subtract(b, a))

    def test_multiplication_distributes_over_addition(self):
        for a, b, c in itertools.product(range(-10, 11), repeat=3):
            with self.subTest(a=a, b=b, c=c):
                self.assertEqual(self.calc.multiply(a, self.calc.add(b, c)),
                                 self.calc.add(self.calc.multiply(a, b), self.calc.multiply(a,
    c)))

    def test_multiply_divide_inverse(self):
        for a, b in itertools.product(range(-25, 26), repeat=2):
            if b == 0:
                continue
            with self.subTest(a=a, b=b):
                recovered = self.calc.divide(self.calc.multiply(a, b), b)
                self.assertTrue(math.isclose(recovered, a, rel_tol=1e-12, abs_tol=1e-12))

    def test_seeded_large_integer_relations(self):
        rng = random.Random(20261025)
        for sample in range(1000):
            a, b, c = (rng.randint(-1_000_000, 1_000_000) for _ in range(3))
            with self.subTest(sample=sample, a=a, b=b, c=c):
                self.assertEqual(self.calc.subtract(self.calc.add(a, b), b), a)
                self.assertEqual(self.calc.subtract(a, b), -self.calc.subtract(b, a))
                self.assertEqual(self.calc.multiply(a, self.calc.add(b, c)),
                                 self.calc.add(self.calc.multiply(a, b), self.calc.multiply(a,
    c)))
                if b != 0:
                    recovered = self.calc.divide(self.calc.multiply(a, b), b)
                    self.assertTrue(math.isclose(recovered, a, rel_tol=1e-12, abs_tol=1e-12))


if __name__ == "__main__":
    unittest.main()
```

Git 提交圖

完整作者、時間、差異統計見隨附 git-log-full.txt。Q2 與 Q7 的獨立 agent 專案另有專屬 Git log 及 bundle。

```
* b890fd7 (HEAD -> main, tag: q3-code-complete, origin/main) Q3: enforce statement and branch coverage in local and cloud checks
*   31333df Integrate reviewed Q6 modulo and power TDD
|\
| * b657173 (tag: q6-complete, origin/codex/q6-tdd, codex/q6-tdd) Q6: preserve review, red-green evidence, and decisions
| * b1b377f Q6 GREEN: implement reviewed modulo and power contract
| * 563359b Q6 RED: commit contract after explicit human review
* |   ada7fed Integrate Q2 CI repair experiment and Q7 tests-only experiment
|\ \
| * | de29f94 (origin/codex/q2-lab, codex/q2-lab) Q2 Q7: preserve experiment inputs, results, CI evidence, and reports
| * | 61036c6 (tag: q2-agent-result) Q2: apply fresh agents blind repair without changing tests
| * | c921aec (tag: q2-broken-again) Q2: reintroduce identical fault for blind repair
| * | a8e5c0c (tag: q2-reference-fixed) Q2: restore subtraction in reference repair
| * | 2f2f59f (tag: q2-broken-first) Q2: introduce fault for first repair
|/ /
* / c6b6caf (tag: q2-ci-green) Q2: run calculator tests in GitHub Actions
|/
* 14ebcc7 (tag: q1-complete) Q1: preserve four TDD cycles and narrative
* 32132ee Q1 GREEN 04: handle division by zero
* b631553 Q1 RED 04: require ValueError for division by zero
* 614df7f Q1 GREEN 03: implement true division
* 1cb2c97 Q1 RED 03: require true division returning float
* 54535ba Q1 GREEN 02: implement multiplication
* 5c11076 Q1 RED 02: define multiplication of integers
* 089ba50 Q1 GREEN 01: implement subtraction
* 9dba640 Q1 RED 01: define subtraction of integers
* 3f22082 Baseline: preserve originals and establish add-only calculator
```